

# Resumo pra prova — HTML e CSS

Base: o projeto **projeto-eventos-2-ciclo** do professor + a atividade que ele disse ser parecida com a prova.

## 1. O esqueleto de todo arquivo HTML

```
<!DOCTYPE html>           ← SEMPRE a 1ª linha. Avisa: "isto é HTML5"
<html lang="pt-BR">        ← idioma da página
<head>                     ← parte INVISÍVEL: configurações
  <meta charset="UTF-8">    ← faz os acentos funcionarem
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
                                ← adapta ao celular
  <title>Página de Eventos</title> ← texto da aba do navegador
  <link rel="stylesheet" href="./assets/css/styles.css"> ← LIGA O CSS
</head>
<body>                     ← parte VISÍVEL: o que aparece na tela
  ...
  <script src="./js/home.js"></script> ← JS no FINAL do body
</body>
</html>
```

- `<!DOCTYPE html>` vem antes de tudo, até antes do `<html>`.
- O CSS se liga com `<link rel="stylesheet" href="...">`, dentro do `<head>`.
- O JavaScript se liga com `<script src="...">` — o professor põe no fim do `<body>` pra rodar só depois que a página carregou.

## 2. Tags semânticas — a "planta baixa" da página

Tag semântica é a que **diz o que o bloco é**, não só como ele aparece. O projeto do professor é montado assim:

```
<nav class="header-menu"> → menu de navegação (logo + links)
<header class="hero">    → cabeçalho visível: título, frase, busca
<section class="events"> → seção de conteúdo: filtros + cards
<footer class="footer">  → rodapé: "© 2026 Eventos Fatec"
```

| Tag                          | O que é                                                 |
|------------------------------|---------------------------------------------------------|
| <code>&lt;header&gt;</code>  | Cabeçalho <b>visível</b> do topo (nome do site, título) |
| <code>&lt;nav&gt;</code>     | Grupo de <b>links de navegação</b> (o menu)             |
| <code>&lt;main&gt;</code>    | Conteúdo principal da página (só um por página)         |
| <code>&lt;section&gt;</code> | Um bloco/capítulo do conteúdo                           |
| <code>&lt;article&gt;</code> | Conteúdo que faz sentido sozinho (post, notícia, card)  |
| <code>&lt;footer&gt;</code>  | Rodapé                                                  |

⚠ **<head> ≠ <header>**. O `<head>` é configuração invisível. O `<header>` é o topo que aparece na tela.

### 3. Link ou botão? <a> x <button>

| Use                                       | Quando                                                              |
|-------------------------------------------|---------------------------------------------------------------------|
| <code>&lt;a href="evento.html"&gt;</code> | Pra <b>ir pra outra página</b> (navegar)                            |
| <code>&lt;button&gt;</code>               | Pra uma <b>ação na própria página</b> (filtrar, abrir menu, enviar) |

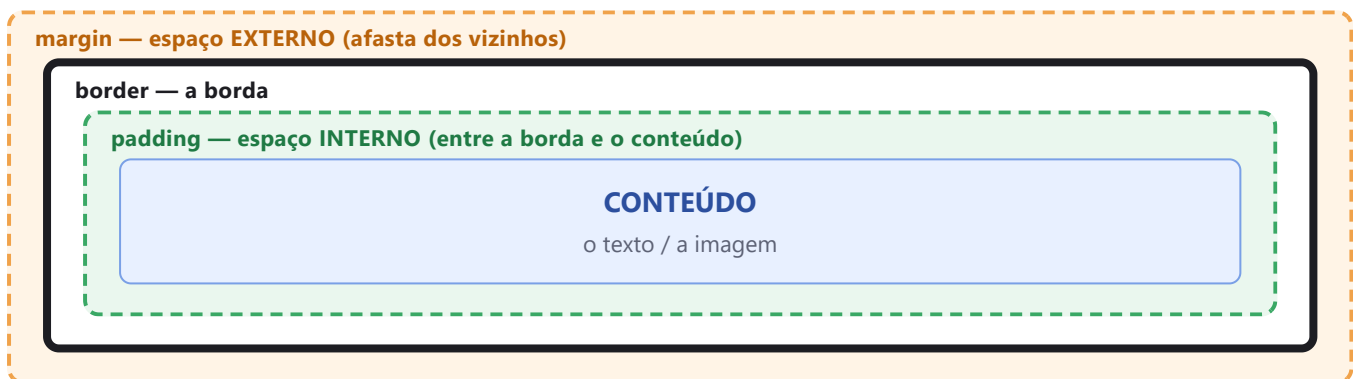
**No projeto:** o "Ver detalhes" de cada card é um `<a>` que o JavaScript cria sozinho:

```
const link = document.createElement("a");
link.href = "evento.html?id=" + evento.id; // leva pra página do evento
link.textContent = "Ver detalhes";
```

O `?id=workshop-1` no fim do endereço diz **qual** evento abrir.

### 4. Box Model — toda caixa tem 4 camadas

De **dentro pra fora**:



- **padding** = dentro da borda
- **margin** = fora da borda

**No projeto**, a 1ª regra do CSS é:

```
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box; /* a largura passa a INCLUIR padding e borda */
}
```

Sem o `border-box`, uma caixa de `width: 300px` com `padding: 20px` ficaria com 340px na tela. Com ele, fica com 300px mesmo.

### 5. Flexbox x Grid — as duas formas de organizar

Os dois são ligados pela **mesma propriedade**: `display`.

|                    | Flexbox                                  | Grid                                 |
|--------------------|------------------------------------------|--------------------------------------|
| <b>Liga com</b>    | <code>display: flex;</code>              | <code>display: grid;</code>          |
| <b>Organiza em</b> | <b>1 direção</b> (linha OU coluna)       | <b>2 direções</b> (linhas E colunas) |
| <b>No projeto</b>  | O <b>menu</b> (logo e links lado a lado) | A <b>lista de cards</b> de eventos   |

## Flexbox — o menu

```
.header-menu {
  display: flex;           /* filhos lado a lado */
  justify-content: center; /* centraliza na HORIZONTAL */
  align-items: center;     /* centraliza na VERTICAL */
  gap: 15px 30px;         /* espaço ENTRE os itens */
  flex-wrap: wrap;        /* se não couber, quebra pra linha de baixo */
}
```

- `justify-content` → alinha no **eixo principal** (numa linha, é a horizontal)
- `align-items` → alinha no **eixo cruzado** (numa linha, é a vertical)
- `flex-direction: column` → empilha em coluna em vez de linha

## Grid — os cards

```
.list {
  display: grid;
  grid-template-columns: repeat(auto-fill, minmax(280px, 1fr));
  gap: 22px;
}
```

Lendo essa linha em português: "crie **quantas colunas couberem**, cada uma com **no mínimo 280px**, e divida o espaço que sobrar **igualmente** entre elas." No computador dá 3 ou 4 cards por linha; no celular, 1 — sem media query nenhuma.

- `fr` = uma **fração** do espaço livre
- `repeat(3, 1fr)` = 3 colunas iguais

## 6. :hover — quando o mouse passa por cima

`:hover` é uma **pseudo-classe**: aplica um estilo só enquanto o cursor está sobre o elemento.

```
.botao:hover {
  background-color: var(--pink-secondary);
}
```

No projeto ele escreve de um jeito mais novo, com CSS aninhado:

```
.link {
  color: var(--text-gray);
  &:hover { color: var(--text-white); } /* & = "este próprio .link" */
}
```

É a mesma coisa que `.link:hover { ... }`.

## 7. Mobile-first e media queries

**Media query** = um bloco de CSS que só vale em certo tamanho de tela.

| Abordagem    | CSS base pensa em... | A media query usa      | Pra...                              |
|--------------|----------------------|------------------------|-------------------------------------|
| Mobile-first | celular              | <code>min-width</code> | acrescentar coisas em telas maiores |

| Abordagem     | CSS base pensa em... | A media query usa | Pra...                           |
|---------------|----------------------|-------------------|----------------------------------|
| Desktop-first | computador           | max-width         | <b>ajustar</b> pra telas menores |

```
/* MOBILE-FIRST: base = celular; a partir de 768px, ajusta pro desktop */
.cards { display: block; }
@media (min-width: 768px) {
  .cards { display: grid; }
}
```

⚠ **PEGADINHA:** o projeto do professor é **desktop-first**. A única media query dele é `@media (max-width: 720px)` — "até 720px", ou seja, ajusta **pro celular**.

Se cair na prova "o que é mobile-first", **responda a definição** (celular primeiro, depois adapta pra telas maiores) — não se guie pelo código do projeto.

## 8. Variáveis CSS

O professor guarda as cores num lugar só:

```
:root {
  --pink-primary: #e20543;
  --bg-dark: #111;
}

.botao { background-color: var(--pink-primary); }
```

- Cria com `--nome` dentro de `:root`
- Usa com `var(--nome)`
- Troca a cor uma vez e muda o site inteiro



## Pegadinhas — como as alternativas erradas enganam

Na atividade, **metade das alternativas erradas é sintaxe inventada**. Se você nunca viu, desconfie.

| Pergunta sobre      | ✓ Certo                                         | ✗ Não existe / é outra coisa                                                                                                |
|---------------------|-------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------|
| Cabeçalho visível   | <code>&lt;header&gt;</code>                     | <code>&lt;head&gt;</code> é invisível · <code>&lt;footer&gt;</code> é rodapé · <code>&lt;h6&gt;</code> é título pequeno     |
| Menu                | <code>&lt;nav&gt;</code>                        | <code>&lt;link&gt;</code> liga CSS · <code>&lt;main&gt;</code> é o conteúdo principal                                       |
| 1ª linha            | <code>&lt;!DOCTYPE html&gt;</code>              | <code>&lt;html&gt;</code> vem <b>depois</b> dele                                                                            |
| Ligar o CSS         | <code>&lt;link rel="stylesheet" href&gt;</code> | <code>&lt;style src&gt;</code> não existe · <code>&lt;script&gt;</code> é pra JS · <code>&lt;css file&gt;</code> não existe |
| Espaço interno      | <code>padding</code>                            | <code>margin</code> é <b>externo</b>                                                                                        |
| Ativar Grid         | <code>display: grid;</code>                     | <code>grid: on</code> · <code>layout: grid</code> · <code>position: grid</code> — inventados                                |
| Ativar Flex         | <code>display: flex;</code>                     | <code>flex: true</code> · <code>align: flex</code> — inventados · <code>display: block</code> empilha                       |
| Mouse em cima       | <code>:hover</code>                             | <code>:click</code> · <code>:mouse</code> · <code>:over</code> — inventados                                                 |
| Mobile-first        | <b>CSS pensando no celular primeiro</b>         | <b>Não</b> é fazer app · <b>não</b> é só celular                                                                            |
| Ir pra outra página | <code>&lt;a href&gt;</code>                     | <code>&lt;button&gt;</code> é ação na página · <code>&lt;link&gt;</code> não é clicável                                     |

**Regra de ouro:** o layout sempre muda pelo `display:` — `display: flex`, `display: grid`, `display: block`, `display: none`.